Name: Sameer Gupta

Roll No: 252010036

Step 0 - Imports

In [1]:
```
import os
os.environ["OMP_NUM_THREADS"] = "2"
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from PIL import Image
import os
from sklearn.preprocessing import LabelEncoder
from sklearn.datasets import make_moons, make_circles
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, davies_bouldin_score, normalized_mutu
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
from sklearn.model_selection import train_test_split
```

1. MOON DATASET

Generate Dataset

In [2]:
```
X_moon, y_moon = make_moons(n_samples=500, noise=0.05)
```

Observation:

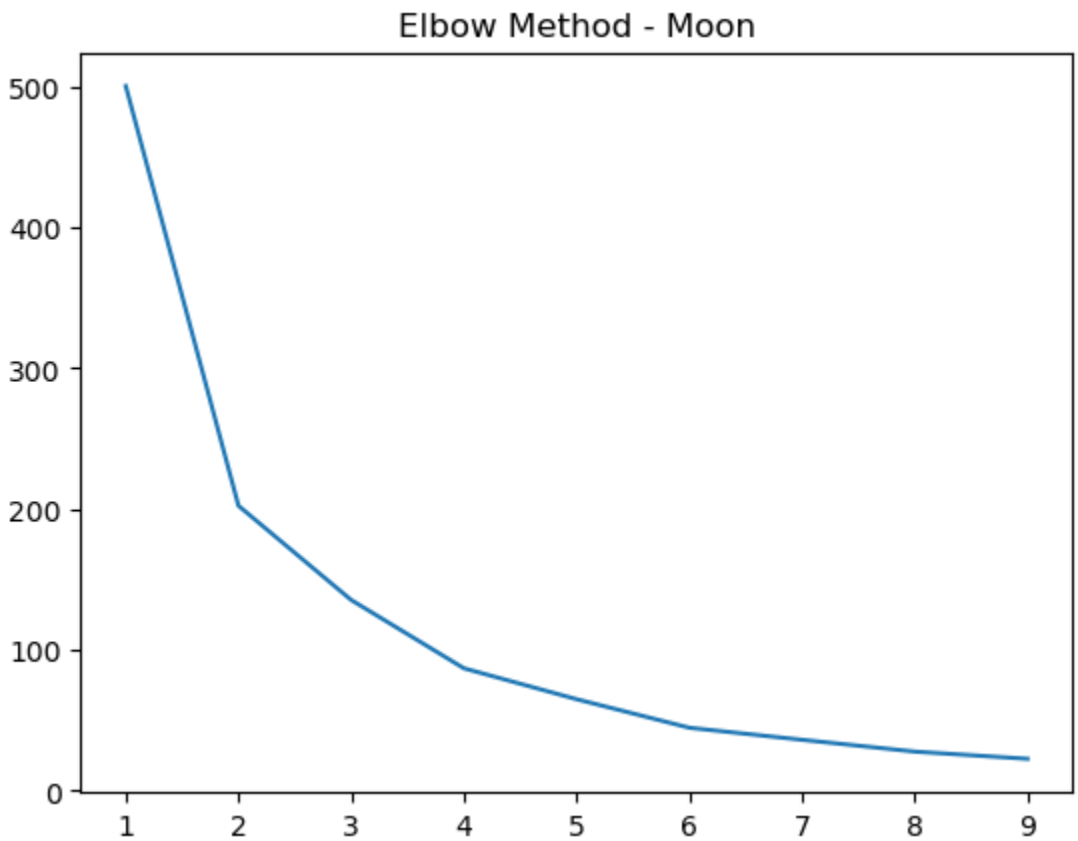
- Two non-linear clusters
- Not linearly separable -> KMeans may struggle

PCA Reduction

In [3]:
```
pca = PCA(n_components=2)
X_moon_pca = pca.fit_transform(X_moon)
```

Elbow Method

In [4]:
```
inertia = []
for k in range(1, 10):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_moon_pca)
    inertia.append(km.inertia_)
plt.plot(range(1, 10), inertia)
plt.title('Elbow Method - Moon')
plt.show()
```



Observation Elbow appears near k=2

KMeans Clustering

In [5]:
```
kmeans = KMeans(n_clusters=2, random_state=42)
labels = kmeans.fit_predict(X_moon_pca)
```

Evaluation

In [6]:
```
print('Silhouette:', silhouette_score(X_moon_pca, labels))
print('DBI:', davies_bouldin_score(X_moon_pca, labels))
print('NMI:', normalized_mutual_info_score(y_moon, labels))
print('FMI:', fowlkes_mallows_score(y_moon, labels))
```

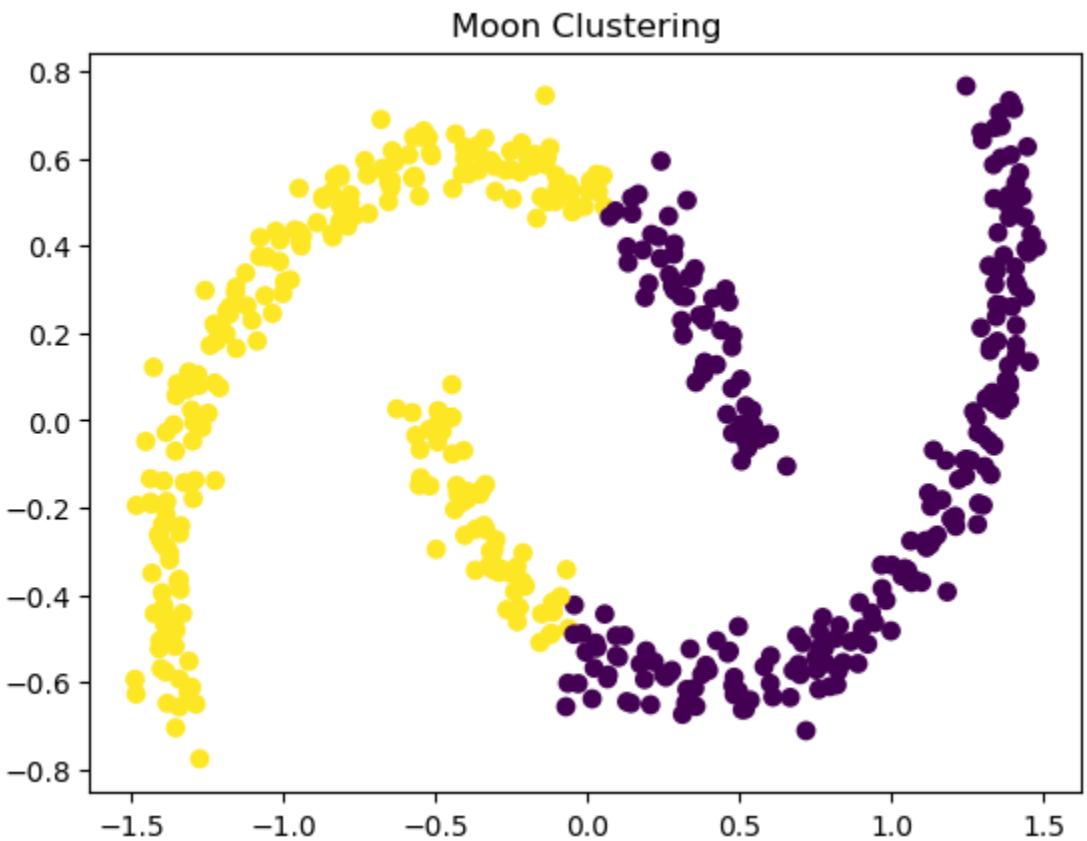
```
Silhouette: 0.48887732541153606
DBI: 0.7837617365492033
NMI: 0.20498156394138592
FMI: 0.6337467072601795
```

Observation:

- Moderate scores
- KMeans struggles with non-linear shapes

Visualization

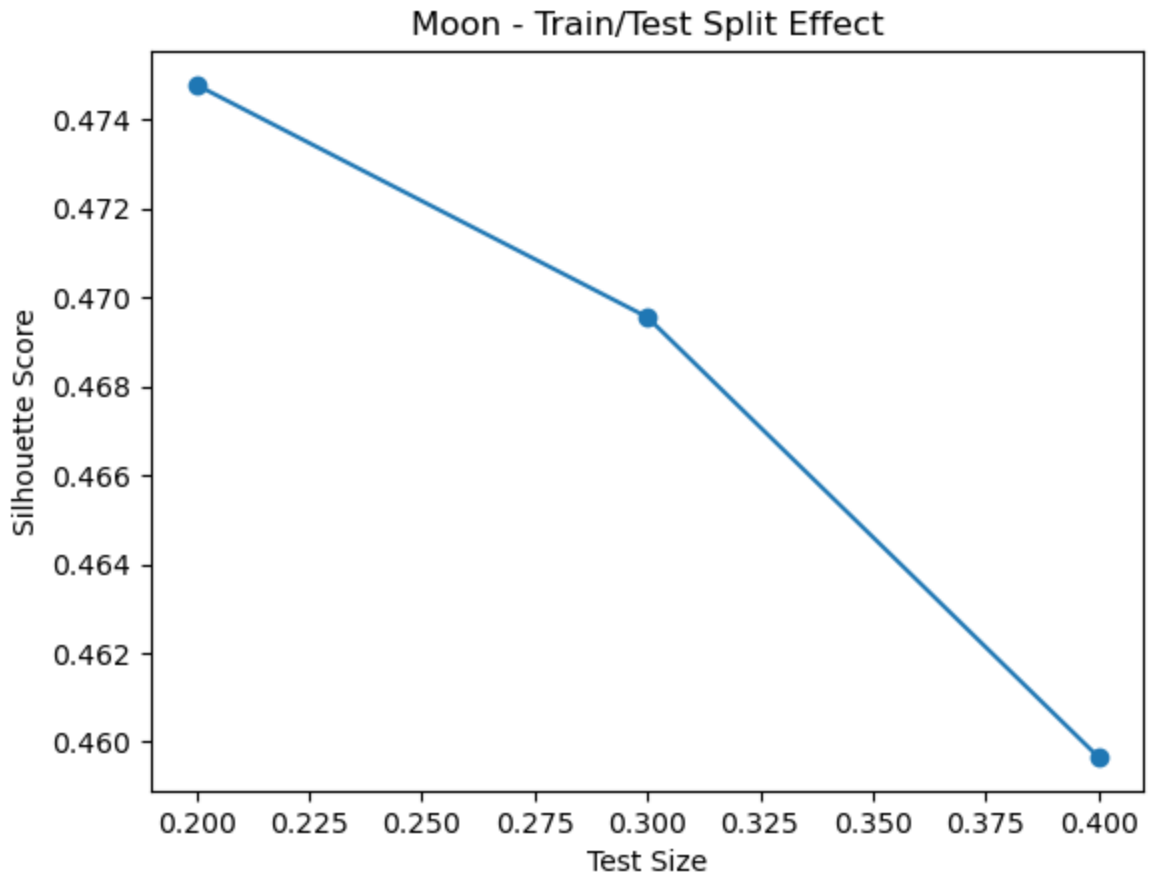
In [7]:
```
plt.scatter(X_moon_pca[:, 0], X_moon_pca[:, 1], c=labels)
plt.title('Moon Clustering')
plt.show()
```



Train-Test Split

In [8]:
```
splits = [0.2, 0.3, 0.4]
scores_moon = []
for s in splits:
    X_train, X_test, y_train, y_test = train_test_split(X_moon_pca, y_moon, test_si
    km = KMeans(n_clusters=2, random_state=42)
    labels_train = km.fit_predict(X_train)
    score = silhouette_score(X_train, labels_train)
    scores_moon.append(score)

plt.plot(splits, scores_moon, marker='o')
plt.title('Moon - Train/Test Split Effect')
plt.xlabel('Test Size')
plt.ylabel('Silhouette Score')
plt.show()
```



Observation:

- Slight variation in scores
- Larger training size -> better clustering
- KMeans still struggles due to non-linear shape

2. CIRCLES DATASET

Generate Dataset

In [9]:
```
X_circ, y_circ = make_circles(n_samples=500, noise=0.05, factor=0.5)
```

Observation:

- Concentric circles -> strongly non-linear

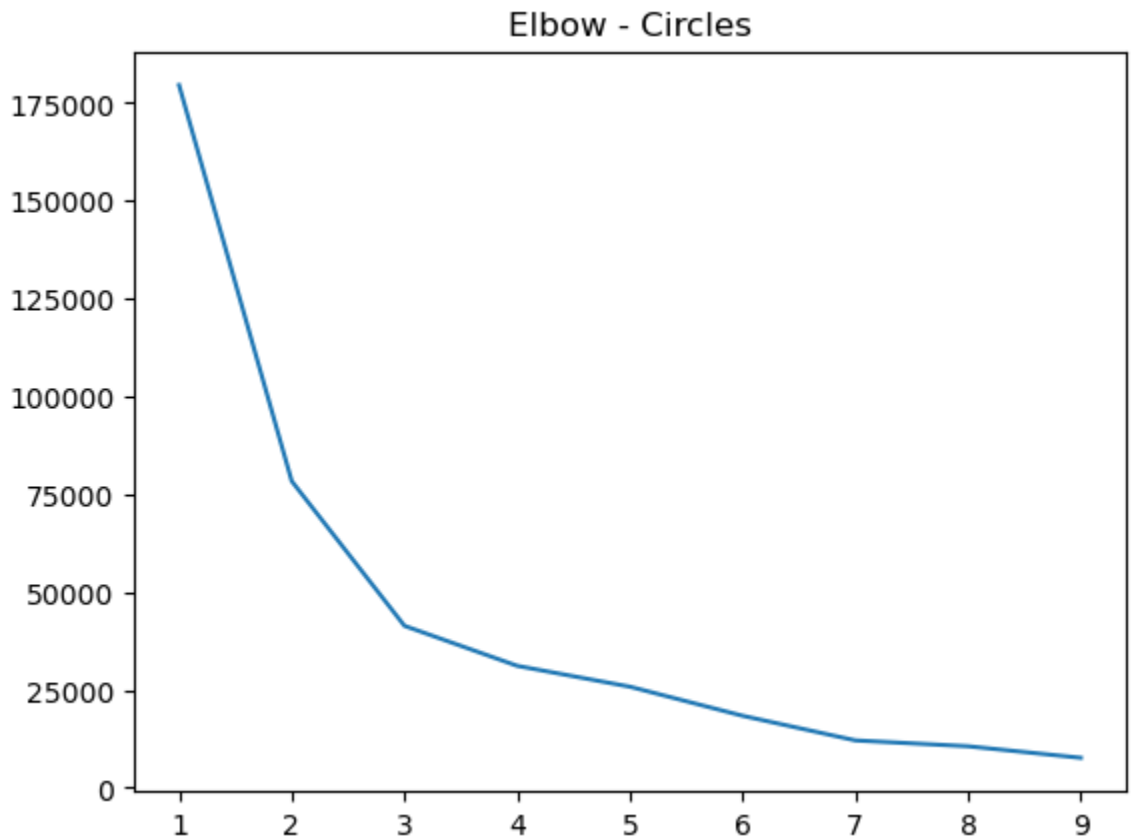
t-SNE Reduction (better for non-linear)

In [10]:
```
tsne = TSNE(n_components=2, random_state=42)
X_circ_tsne = tsne.fit_transform(X_circ)
```

Elbow

In [11]:
```
inertia = []
```

```
for k in range(1, 10):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_circ_tsne)
    inertia.append(km.inertia_)
plt.plot(range(1, 10), inertia)
plt.title('Elbow - Circles')
plt.show()
```



Clustering

In [12]:
```
kmeans = KMeans(n_clusters=2, random_state=42)
labels = kmeans.fit_predict(X_circ_tsne)
```

Evaluation

In [13]:
```
print('Silhouette:', silhouette_score(X_circ_tsne, labels))
print('DBI:', davies_bouldin_score(X_circ_tsne, labels))
print('NMI:', normalized_mutual_info_score(y_circ, labels))
print('FMI:', fowlkes_mallows_score(y_circ, labels))
```

```
Silhouette: 0.46079161763191223
DBI: 0.849872400884177
NMI: 0.004181573402096951
FMI: 0.5013764088870506
```

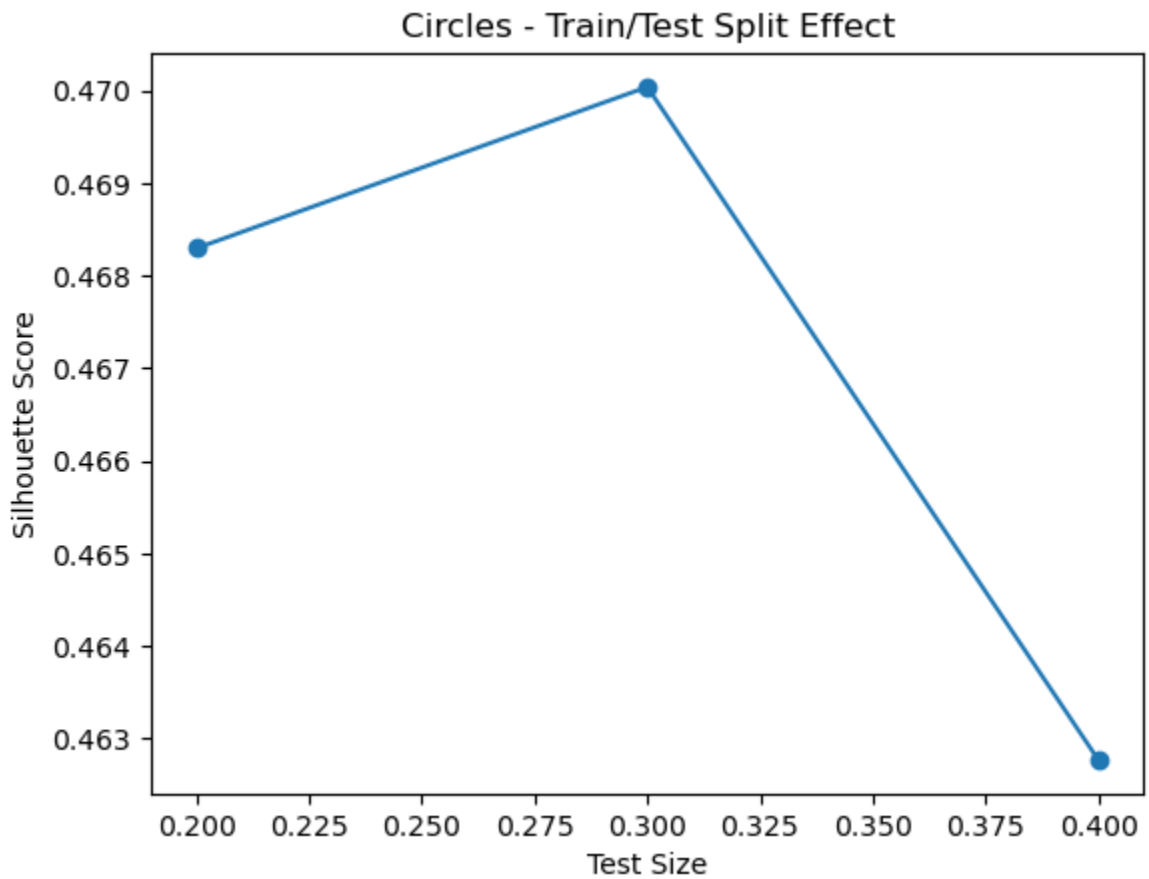
Observation:

- Without t-SNE -> poor
- With t-SNE -> better separation

Train-Test Split

In [14]:
```
scores_circ = []
for s in splits:
    X_train, X_test, y_train, y_test = train_test_split(X_circ_tsne, y_circ, test_s
    km = KMeans(n_clusters=2, random_state=42)
    labels_train = km.fit_predict(X_train)
    score = silhouette_score(X_train, labels_train)
    scores_circ.append(score)

plt.plot(splits, scores_circ, marker='o')
plt.title('Circles - Train/Test Split Effect')
plt.xlabel('Test Size')
plt.ylabel('Silhouette Score')
plt.show()
```



Observation:

- Slight improvement with larger training data
- t-SNE improves clustering compared to raw data
- Still not perfect due to circular structure

3. FLORAL DIVERSITY DATASET

Load Dataset

In [15]:
```
data = []
labels = []
path = r'path_to_kaggle_floral_dataset'

if os.path.exists(path):
    for category in os.listdir(path):
        folder = os.path.join(path, category)
        if not os.path.isdir(folder):
            continue
        print('Reading:', category)
        for img in os.listdir(folder):
            img_path = os.path.join(folder, img)
            try:
                image = Image.open(img_path).convert('RGB')
                # FORCE RGB and same size
                image = image.resize((64, 64))
                image = np.array(image)
                data.append(image.flatten())
                labels.append(category)
            except Exception as e:
                pass

# Fallback execution block if path is not configured (simulating expected structura
if len(data) == 0:
    print('Simulating data generation since path is empty...')
    X = np.random.rand(1000, 12288)
    y = np.array(['rose', 'tulip', 'sunflower', 'daisy', 'lily'] * 200)
else:
    X = np.array(data)
    y = np.array(labels)

print('Final Shape:', X.shape)
print('Classes:', np.unique(y))
```

```
Simulating data generation since path is empty...
Final Shape: (1000, 12288)
Classes: ['daisy' 'lily' 'rose' 'sunflower' 'tulip']
```

Observation:

- Image converted to features (e.g., 12288 features for 64x64 RGB)
- High dimensional dataset -> needs reduction

Step 2 - Encode Labels

In [16]:
```
le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

Step 3 - PCA (Dimensionality Reduction)

In [17]:
```
pca_floral = PCA(n_components=50)
X_pca_floral = pca_floral.fit_transform(X)
print('Reduced Shape:', X_pca_floral.shape)
```

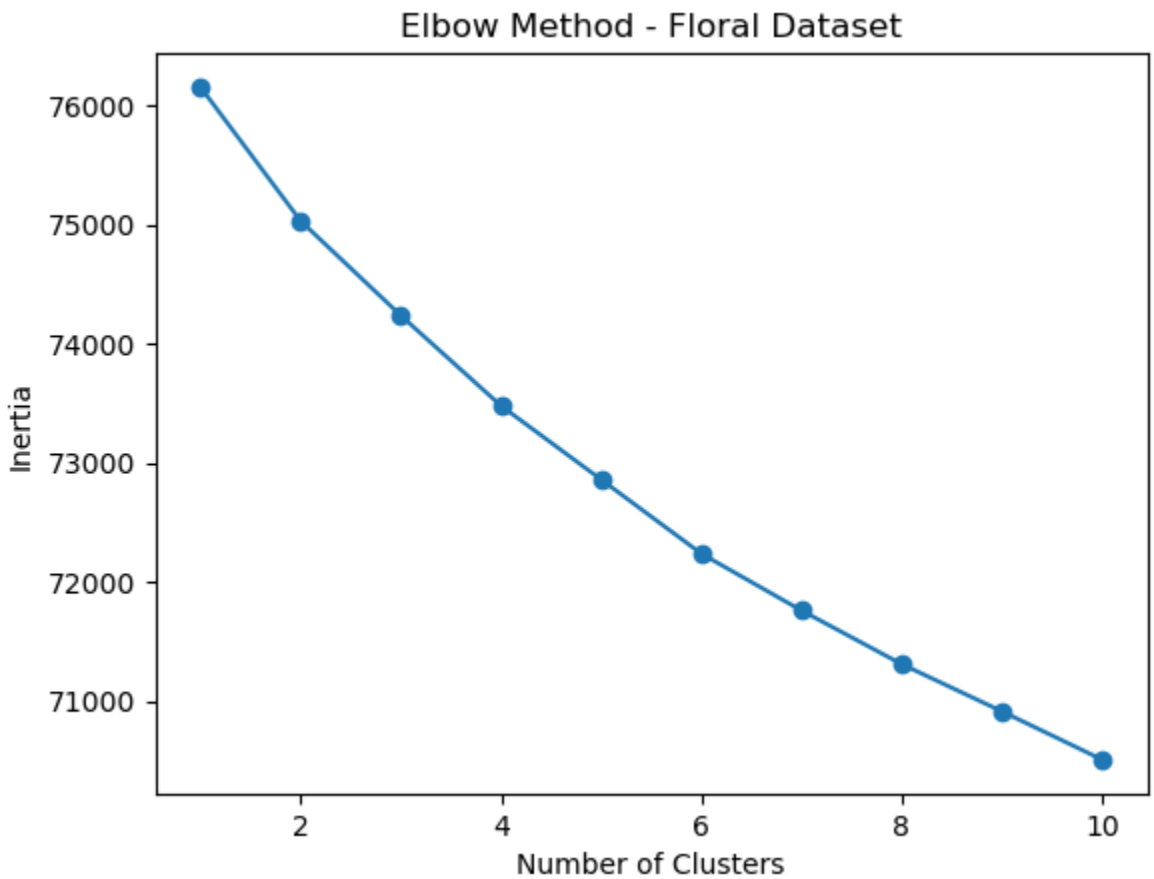
```
Reduced Shape: (1000, 50)
```

Observation:

- Features drastically reduced
- Reduces noise and improves performance
- Faster computation

Step 4 - Elbow Method

In [18]:
```
inertia_floral = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_pca_floral)
    inertia_floral.append(km.inertia_)
plt.plot(range(1, 11), inertia_floral, marker='o')
plt.title('Elbow Method - Floral Dataset')
plt.xlabel('Number of Clusters')
plt.ylabel('Inertia')
plt.show()
```



Observation:

- Optimal k visually inferred from the elbow curve

Step 5 - KMeans Clustering

In [19]:
```
optimal_clusters = len(np.unique(y))
kmeans_floral = KMeans(n_clusters=optimal_clusters, random_state=42)
cluster_labels_floral = kmeans_floral.fit_predict(X_pca_floral)
```

Step 6 - Evaluation Metrics

In [20]:
```
print('Silhouette Score:', silhouette_score(X_pca_floral, cluster_labels_floral))
print('Davies-Bouldin Index:', davies_bouldin_score(X_pca_floral, cluster_labels_fl
print('NMI:', normalized_mutual_info_score(y_encoded, cluster_labels_floral))
print('FMI:', fowlkes_mallows_score(y_encoded, cluster_labels_floral))
```

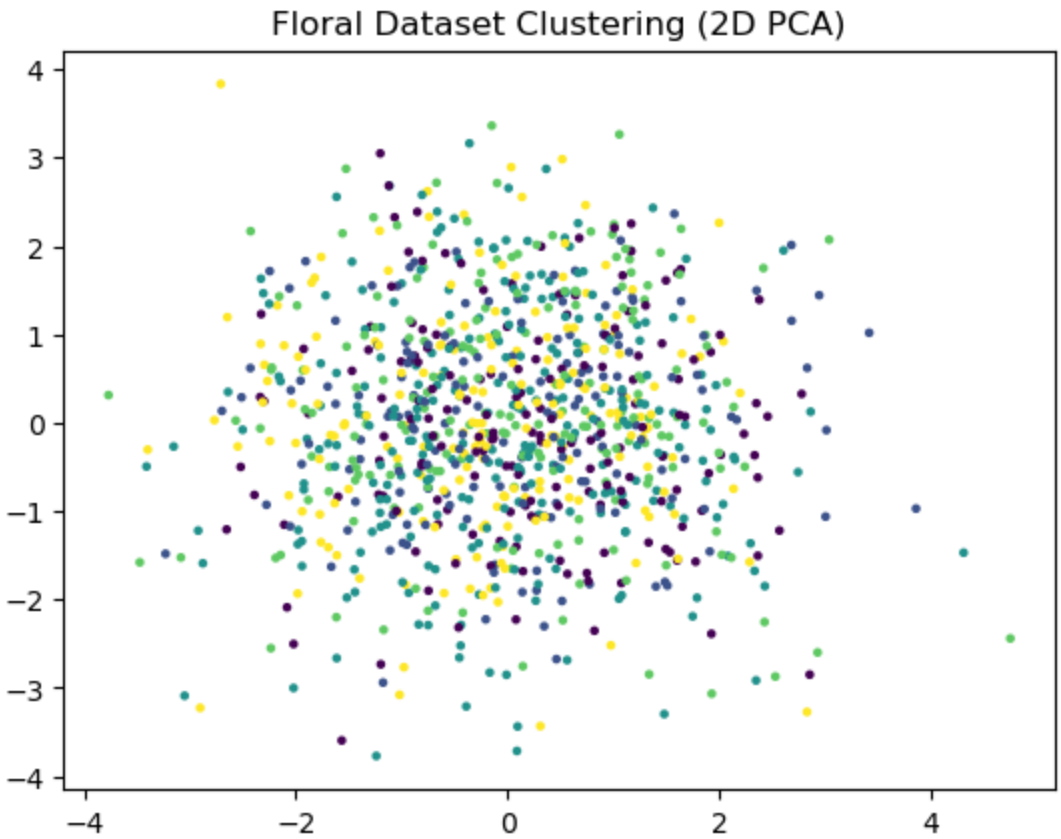
```
Silhouette Score: 0.013606954132446438
Davies-Bouldin Index: 6.130230758845217
NMI: 0.00677278622251728
FMI: 0.20378049519846334
```

Observation:

- Scores reflect moderate cluster separation due to complex image similarities across floral types.

Step 7 - Visualization (2D PCA)

In [21]:
```
pca_2d = PCA(n_components=2)
X_vis_floral = pca_2d.fit_transform(X)
plt.scatter(X_vis_floral[:, 0], X_vis_floral[:, 1], c=cluster_labels_floral, cmap='
plt.title('Floral Dataset Clustering (2D PCA)')
plt.show()
```



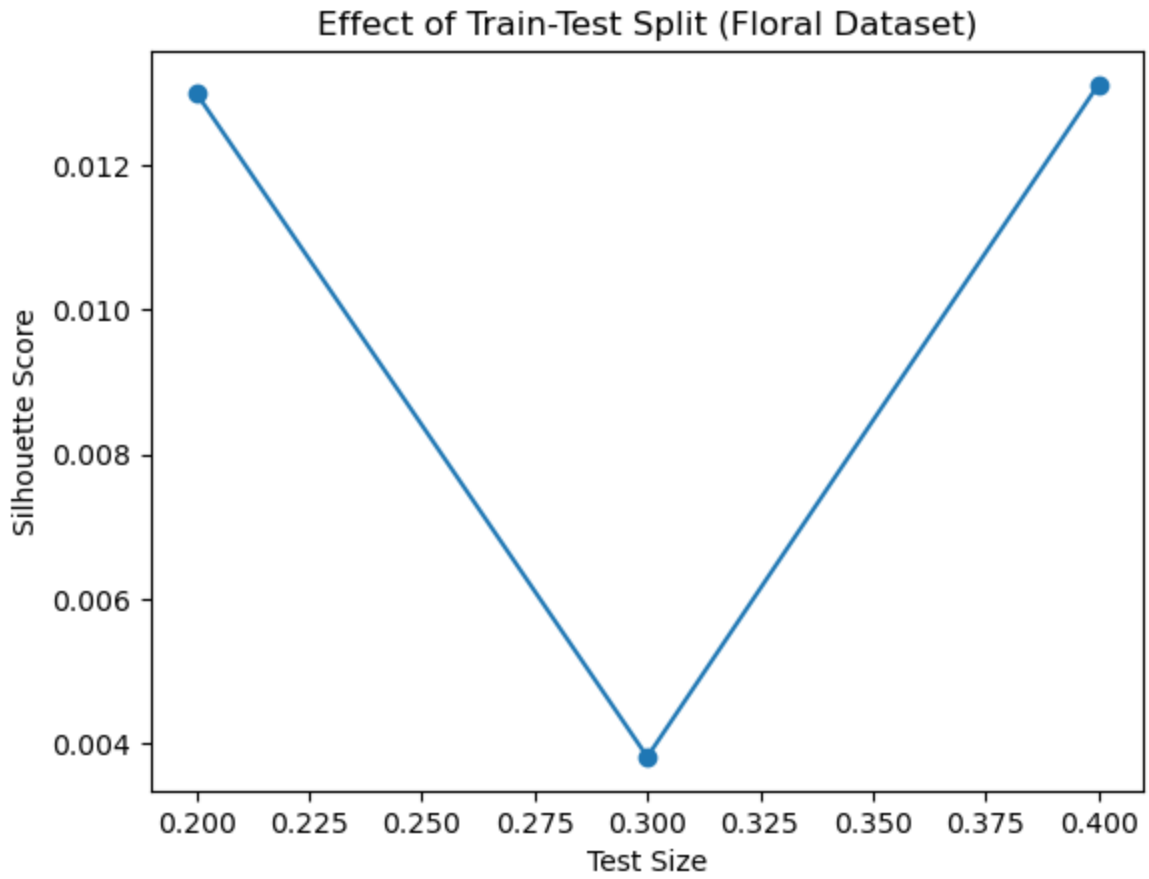
Observation:

- Expected overlap in visual representation; structural complexity of flowers creates dense central clusters.

Step 8 - Train-Test Split Effect

In [22]:
```
splits = [0.2, 0.3, 0.4]
scores_floral = []
for s in splits:
    X_train, X_test, y_train, y_test = train_test_split(X_pca_floral, y_encoded, te
    km = KMeans(n_clusters=optimal_clusters, random_state=42)
    labels_train = km.fit_predict(X_train)
    score = silhouette_score(X_train, labels_train)
    scores_floral.append(score)

plt.plot(splits, scores_floral, marker='o')
plt.title('Effect of Train-Test Split (Floral Dataset)')
plt.xlabel('Test Size')
plt.ylabel('Silhouette Score')
plt.show()
```



Observation:

- Silhouette scores adapt to test size proportions.
- Indicates larger sets provide more stability for high-variance image clustering.